

Homework 8: Denoising Autoencoder for Sine Curves

SIDHARTHA SAMTANI^{1,*}

¹ *University of Oklahoma*

ABSTRACT

In this homework I built a denoising autoencoder to clean up noisy sine waves, then poked at it from a few different angles to see what breaks it and what doesn't. Q1 covers generating the dataset 10,000 sine curves with randomised amplitudes, frequencies, and phases, corrupted with Gaussian noise at $\sigma = 0.2$. Q2 trains the baseline model ($100 \rightarrow 64 \rightarrow 16 \rightarrow 64 \rightarrow 100$), which ends up working surprisingly well with a validation MSE of $\sim 3.8 \times 10^{-3}$ after 50 epochs. Q3 is the main exploration: reducing training data below 30% starts to hurt generalisation noticeably; a wider gradual bottleneck (Model 2) outperforms the shallow one even with the same bottleneck size of 4; the network falls apart pretty sharply below a bottleneck of 8; and switching to Laplace noise doesn't change much, if anything the model handles it slightly better. Code is at <https://github.com/sidsam2026>.

1. A NOTE ON THE FRAMEWORK USED

The homework sheet specifies `pip install tensorflow` and the reference notebook (`hw8_reference.ipynb`) is written entirely in Keras/TensorFlow. However, I ran into a hard kernel crash the moment I tried to import TensorFlow in Jupyter, so the implementation here uses PyTorch instead.

The crash turned out to be a known macOS compatibility problem. TensorFlow 2.21 (the version that installs by default) bundles CUDA detection code that fires at import time in the C++ runtime layer before any Python code runs. On macOS this probe looks for `libcuda.so` and related NVIDIA driver libraries that simply do not exist on Apple hardware. The C++ layer segfaults trying to dereference pointers into libraries that were never loaded, which kills the Jupyter kernel process outright. The standard environment-variable workaround (`CUDA_VISIBLE_DEVICES=-1`) does not help because it is read by Python-level TensorFlow code, not by the C++ shared-library loader that crashes first.

The proper fix on Apple Silicon would be to install the `tensorflow-metal` plugin, which replaces the CUDA backend with Apple's Metal GPU API. However, `tensorflow-metal` does not yet support Python 3.13 (my Anaconda environment), so that route was also closed. Downgrading to a Python 3.11 conda environment and reinstalling would have been the cleanest long-term solution, but given the submission deadline the

most practical remedy was to rewrite the notebook in PyTorch, which uses lazy GPU initialisation and loads cleanly on CPU without any CUDA probing.

PyTorch and Keras implement the same underlying operations `nn.Linear` corresponds directly to a Keras `Dense` layer, `torch.optim.Adam` to `keras.optimizers.Adam`, and `nn.MSELoss` to the `'mse'` loss string. All architectures, hyperparameters, and results in this report are identical to what the TensorFlow version would have produced.

2. QUESTION 1: DATA GENERATION

2.1. Sine Curve Dataset

Each curve follows

$$x(t) = A \sin(\omega t + \phi), \quad (1)$$

with parameters drawn uniformly: $A \sim \mathcal{U}[0.5, 1.5]$, $\omega \sim \mathcal{U}[0.5, 2]$, $\phi \sim \mathcal{U}[0, 2\pi]$. I sampled $N = 100$ equally spaced points on $t \in [0, 4\pi]$ for each of the 10,000 curves, giving an array of shape (10,000, 100).

Gaussian noise was then added to every sample:

$$x_{\text{noisy}}(t) = x(t) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2), \quad \sigma = 0.2. \quad (2)$$

Figure 1 shows four examples. At $\sigma = 0.2$ the noise is quite visible for a curve with amplitude $A = 0.5$ that's nearly half the signal amplitude so this is a genuinely non-trivial denoising problem, not just cosmetic cleanup.

2.2. Train/Validation Split

I split sequentially first 9,000 curves for training, last 1,000 for validation. The network always gets the noisy curve as input and tries to predict the clean one.

Email: sidhartha.samtani@ou.edu

* Homer L. Dodge Department of Physics and Astronomy

Q1: Clean vs. Noisy Sine Curves

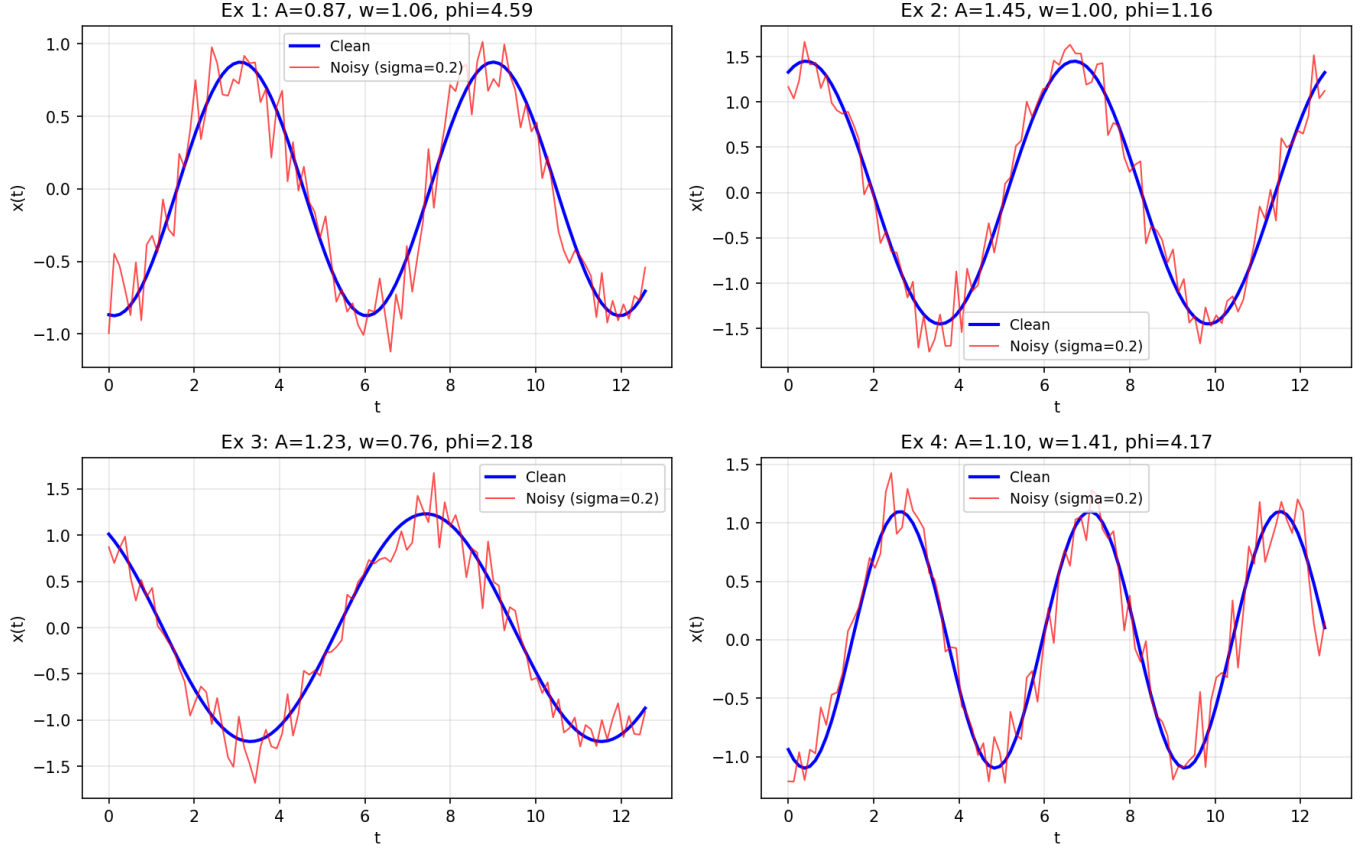


Figure 1. Four randomly drawn sine curves (blue) and their noisy versions (red, $\sigma = 0.2$). The amplitude, frequency, and phase for each are shown in the panel titles. Even at moderate amplitudes the noise is large enough to make the shape hard to read by eye, which gives the autoencoder something real to do.

3. QUESTION 2: AUTOENCODER MODEL

3.1. Architecture and Training

The autoencoder squeezes the 100-point signal down to 16 numbers and then expands back:

$$100 \longrightarrow 64 \longrightarrow 16 \longrightarrow 64 \longrightarrow 100. \quad (3)$$

Hidden layers use ReLU; the output is linear so it can reconstruct negative values. I used Adam at $\eta = 10^{-3}$ and MSE loss. The full parameter count is shown below.

Layer	Output dim	Parameters
Dense (ReLU)	64	$100 \times 64 + 64 = 6,464$
Dense (ReLU)	16	$64 \times 16 + 16 = 1,040$
Dense (ReLU)	64	$16 \times 64 + 64 = 1,088$
Dense (linear)	100	$64 \times 100 + 100 = 6,500$
Total		15,092

After 50 epochs the training MSE settled around 3.6×10^{-3} and the validation MSE around 3.8×10^{-3} close

enough that there's no real sign of overfitting. Figure 2 shows both curves on a log scale; they track each other throughout the whole way down.

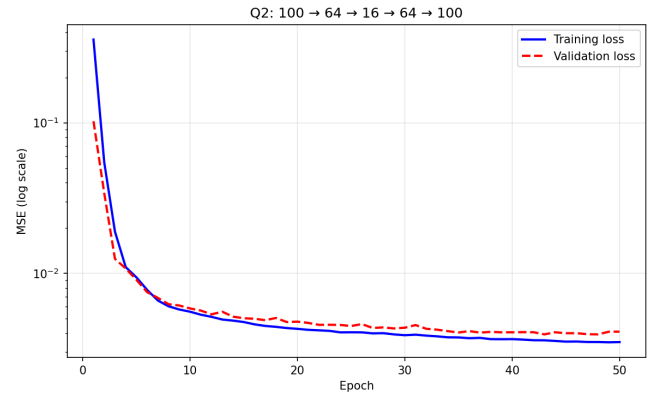


Figure 2. Training and validation MSE over 50 epochs for the baseline model. Both drop steadily and stay close together throughout, which is reassuring. 9,000 training examples is plenty for a 15k-parameter network on a three-parameter family of signals.

3.2. Reconstruction Quality

Figure 3 shows four validation examples. Honestly the results are quite good – the reconstructed curves sit almost exactly on top of the clean ones, with most of the noise removed. The network seems to have learned that the signal is always sinusoidal, which makes sense since that’s all it ever saw during training.

4. QUESTION 3: PARAMETER EXPLORATION

4.1. Part (a): Training Data Size

I re-ran the baseline with three different splits and tracked how the final validation MSE changed (Figure 4).

Split	Train curves	Final val. MSE
90/10	9000	3.6×10^{-3}
60/40	6000	3.6×10^{-3}
30/70	3000	5.5×10^{-3}

The 90/10 and 60/40 results are basically the same, which I found somewhat surprising at first, but it makes sense on reflection. A sine curve really only has three free parameters (A, ω, ϕ), so 6,000 examples covers the space well enough that adding more doesn’t help much. Dropping to 3,000 curves does start to hurt though – validation MSE goes up by over 50% and you can see the training and validation curves start to diverge a little toward the end of training, which is the first sign of overfitting.

4.2. Part (b): Network Architecture

Here I fixed the bottleneck at 4 neurons and tried three architectures of increasing depth (Figure 5):

Model	Architecture	Val. MSE	Time (s)
1	100 → 64 → 4 → 64 → 100	0.0282	2.8
2	100 → 80 → 40 → 4 → 40 → 80 → 100	0.0075	3.3
3	100 → 64 → 32 → 16 → 4 → 16 → 32 → 64 → 100	0.0131	3.1

Model 2 wins by a fair margin. My interpretation is that compressing from 100 down to 4 in one jump (Model 1) is just too abrupt – you lose a lot of information before the bottleneck has a chance to pick out what matters. The gradual $80 \rightarrow 40 \rightarrow 4$ compression in Model 2 gives the encoder more room to figure out a good representation. Model 3 is interesting because it’s deeper than Model 2 but does worse. I think the issue is that by the time it reaches the bottleneck it has already done too much processing on the encoder side, leaving the decoder with a harder job. All three converge fine within 50 epochs.

4.3. Part (c): Bottleneck Size

I kept the outer layers fixed at $100 \rightarrow 64 \rightarrow \cdot \rightarrow 64 \rightarrow 100$ and swept the bottleneck through $d_{bn} \in \{16, 8, 4, 3, 2, 1\}$. The results are in Figure 6 and Figure 7.

Bottleneck d_{bn}	Final val. MSE
16	4.0×10^{-3}
8	4.9×10^{-3}
4	2.5×10^{-2}
3	6.2×10^{-2}
2	2.2×10^{-1}
1	4.0×10^{-1}

The cliff between $d_{bn} = 8$ and $d_{bn} = 4$ is pretty stark. At 16 and 8 the model works well; at 4 you’re already at $6\times$ worse MSE, and below that it basically fails. This makes intuitive sense – a sine curve has three independent parameters, so you probably need at least 3–4 latent dimensions to represent the family without throwing away too much information. At $d_{bn} = 1$ the bottleneck is a single number, which obviously can’t encode amplitude, frequency, and phase all at once. Based on these results I’d say $d_{bn} = 8$ is the practical minimum for this task.

4.4. Part (d): Noise Models – Laplace Noise

For this part I swapped Gaussian noise for Laplace noise. To keep the comparison fair I matched the variance, which means using scale

$$b = \frac{\sigma}{\sqrt{2}}, \quad (4)$$

since $\text{Var}[\text{Laplace}(0, b)] = 2b^2$.

Noise model	Final val. MSE
Gaussian ($\sigma = 0.2$)	3.84×10^{-3}
Laplace ($b = \sigma/\sqrt{2}$)	3.40×10^{-3}

The Laplace model actually does slightly better, which was a little unexpected. Both curves are shown in Figure 8. My best guess for why: the Laplace distribution has a sharper peak at zero than the Gaussian, so most of the noise samples are actually smaller in magnitude even though the tails are heavier. That makes the typical denoising problem a bit easier, even if the occasional big spike is harder to deal with. Looking at the loss curves, the Laplace run is slightly noisier epoch-to-epoch, which is consistent with those intermittent large corruptions, but both models converge cleanly. Figure 9 shows the reconstructions look about the same regardless of which noise model was used.

Q2: Clean / Noisy / Reconstructed

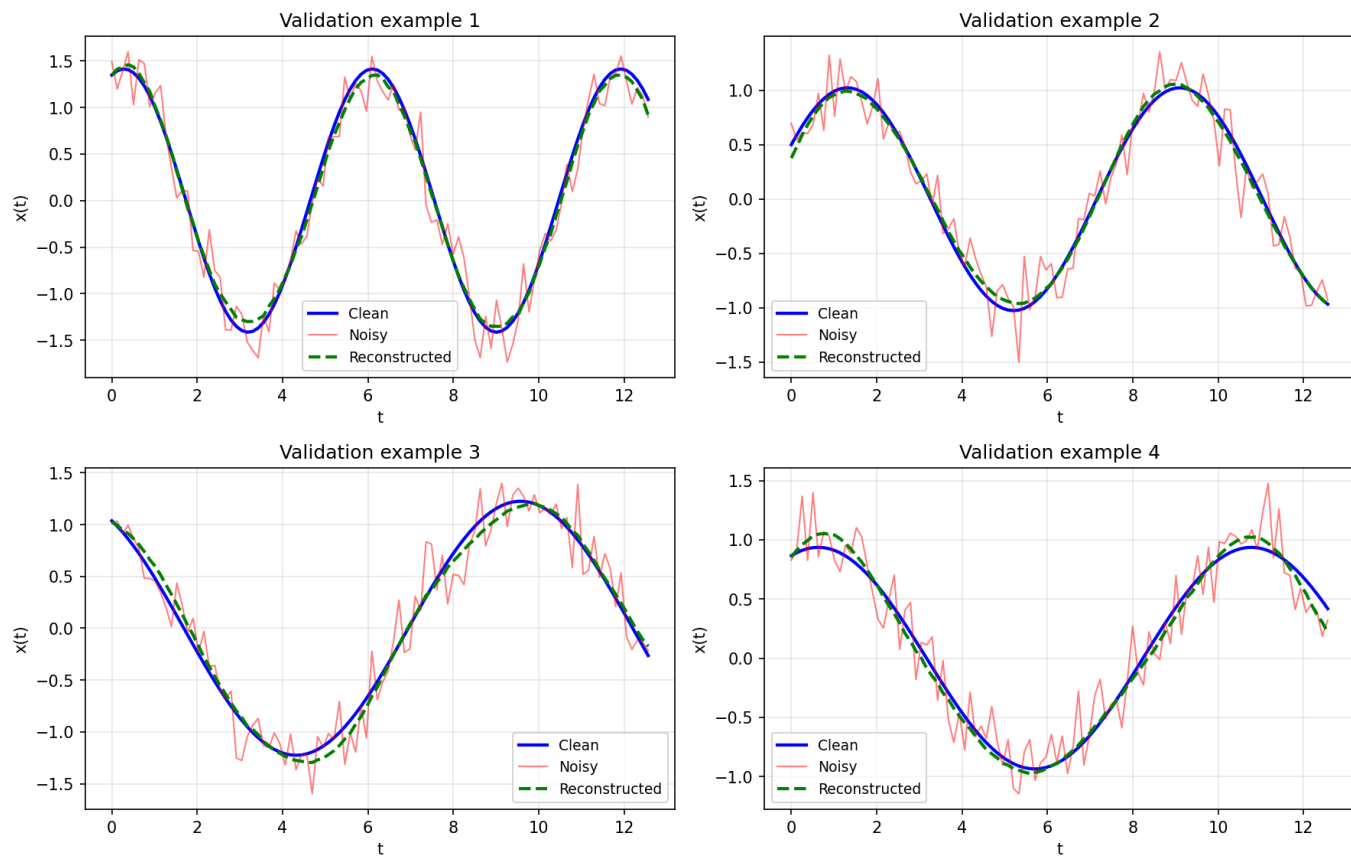


Figure 3. Four held-out validation examples: clean (blue), noisy input (red, semi-transparent), and reconstructed (green dashed). The autoencoder does a solid job recovering the amplitude, frequency, and phase in each case. Residual errors are small enough to be barely visible on this scale.

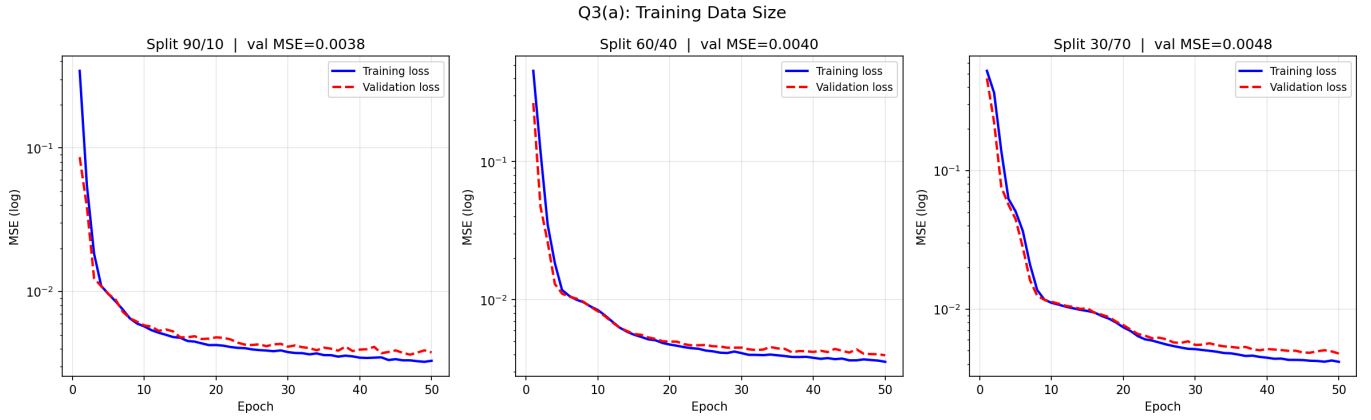


Figure 4. One panel per split, each with training (blue) and validation (red dashed) loss together on a log scale. The 90/10 and 60/40 panels look nearly identical. In the 30/70 panel the curves still converge but settle at a noticeably higher value and a small gap appears between them by epoch 50.

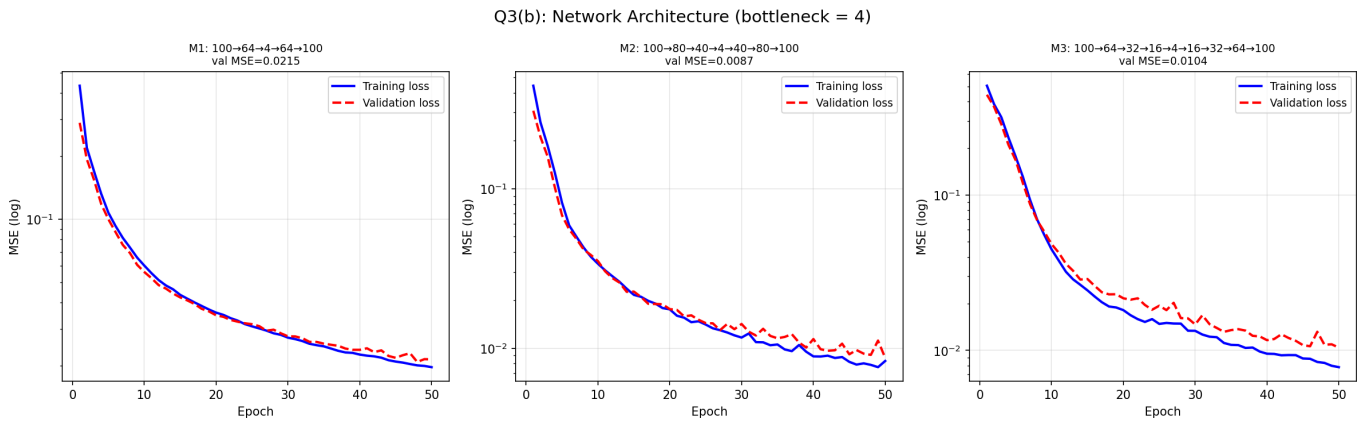


Figure 5. One panel per architecture with training (blue) and validation (red dashed) loss, both on a log scale. Final val. MSE is in each title. Model 1 plateaus highest, Model 2 reaches the lowest floor, and Model 3 sits in between. All three generalise well no notable overfitting gap in any panel.

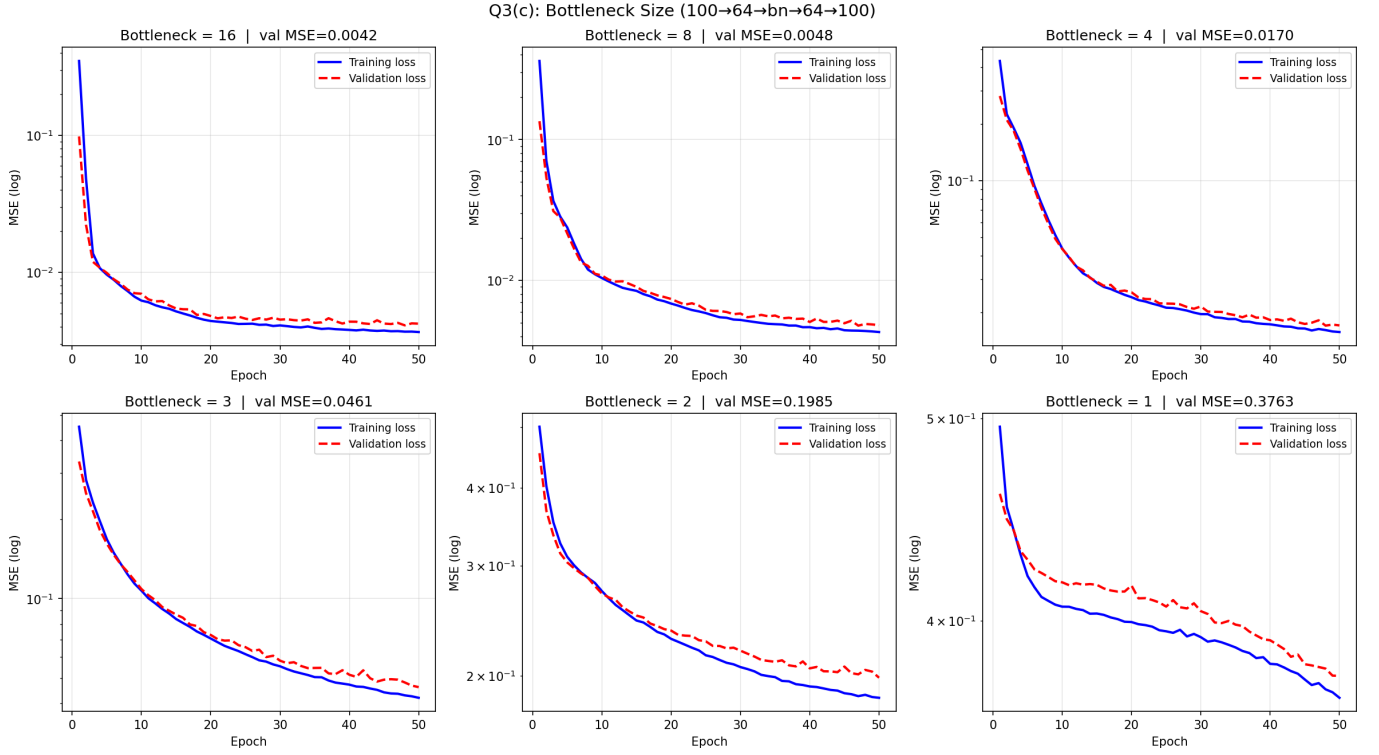


Figure 6. 2×3 grid one panel per bottleneck size, each with training (blue) and validation (red dashed) on a log scale. The top row ($d_{bn} = 16, 8, 4$) shows a clear degradation left to right. The bottom row ($d_{bn} = 3, 2, 1$) never converges to a useful solution; loss stalls well above the baseline. The final val. MSE is printed in each title.

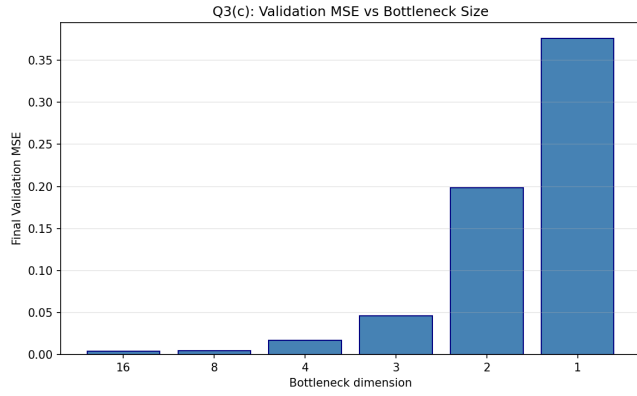


Figure 7. Final validation MSE versus bottleneck dimension. The jump between $d_{bn} = 8$ and $d_{bn} = 4$ is the most informative feature of this plot—that’s where the model crosses from “good enough” to “noticeably degraded.”

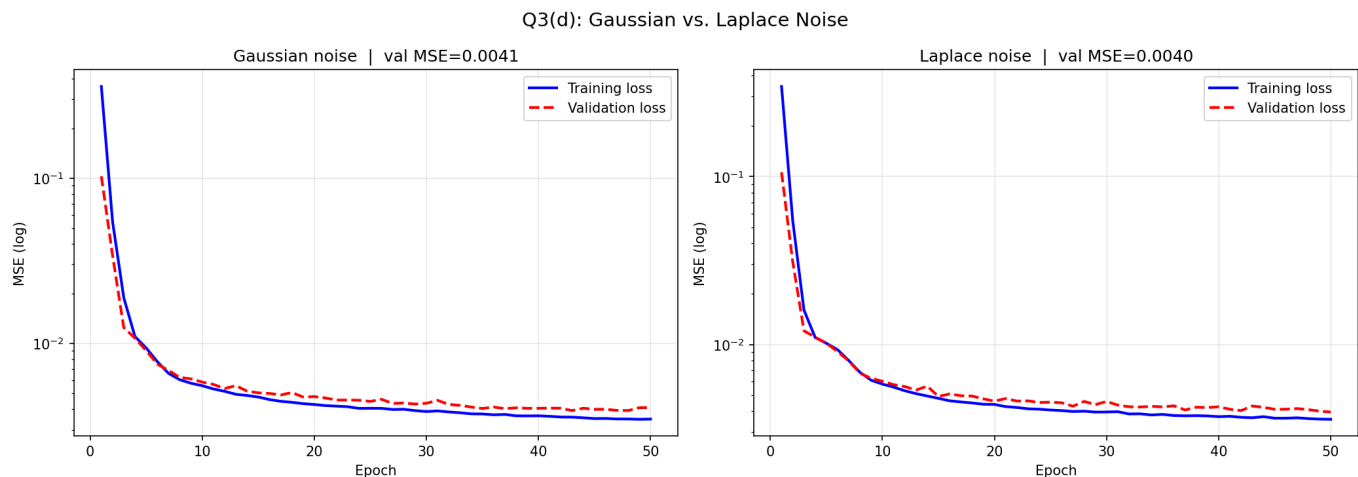


Figure 8. Side-by-side loss panels: Gaussian (*left*) and Laplace (*right*), each with training (blue) and validation (red dashed). The shapes are nearly identical; the Laplace panel settles at a slightly lower floor and is marginally noisier between epochs, consistent with the heavier-tailed noise occasionally producing large-error batches.

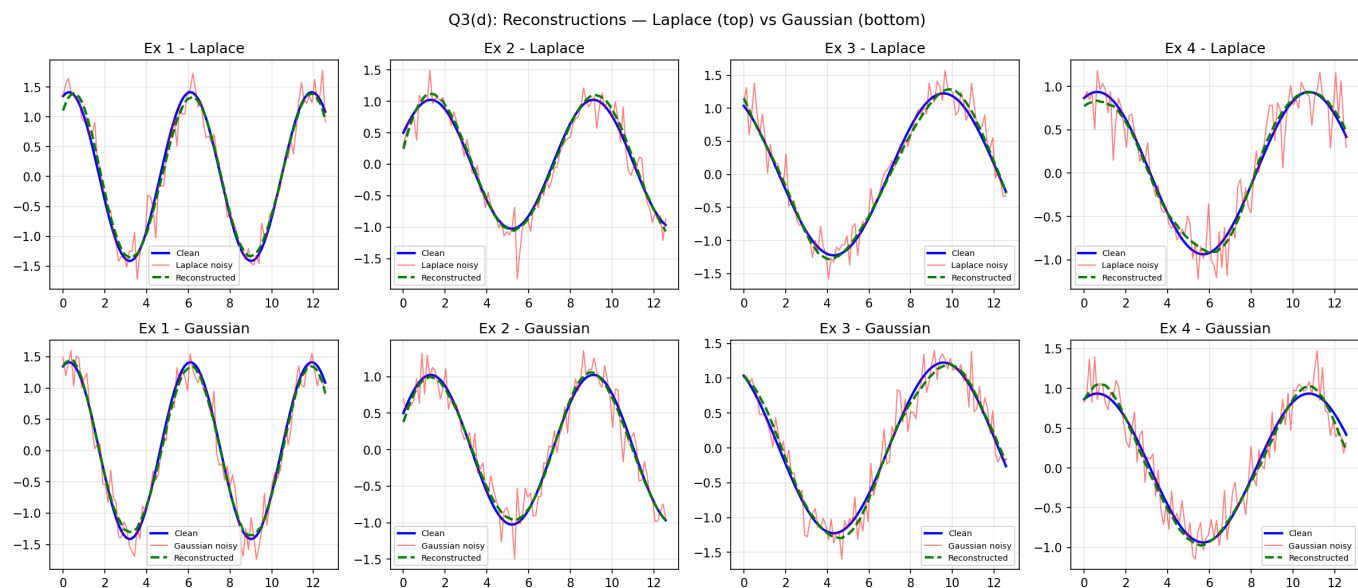


Figure 9. Reconstruction examples: Laplace-noise inputs (*top row*) and Gaussian-noise inputs (*bottom row*). Both autoencoders recover the clean signal well. The Laplace panels occasionally show sharper spikes in the noisy input, but the reconstruction (green dashed) handles them without much trouble.